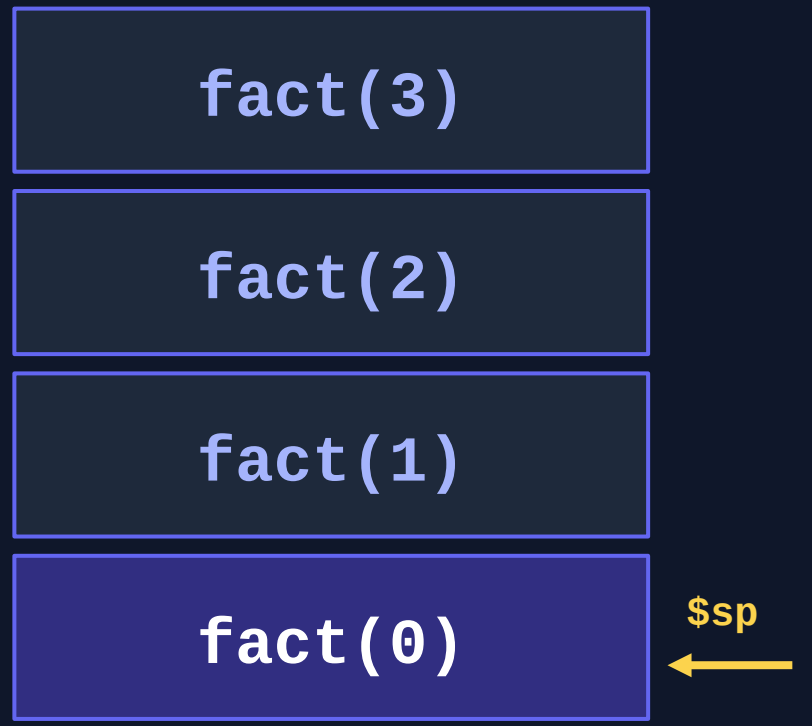
ENCONTRO 5 DE 6

Funções, e de C para MIPS

Curso Introdutório de Programação de Baixo Nível



Liga ProtoFPGA · UFSC Araranguá

O mesmo trecho, escrito duas vezes

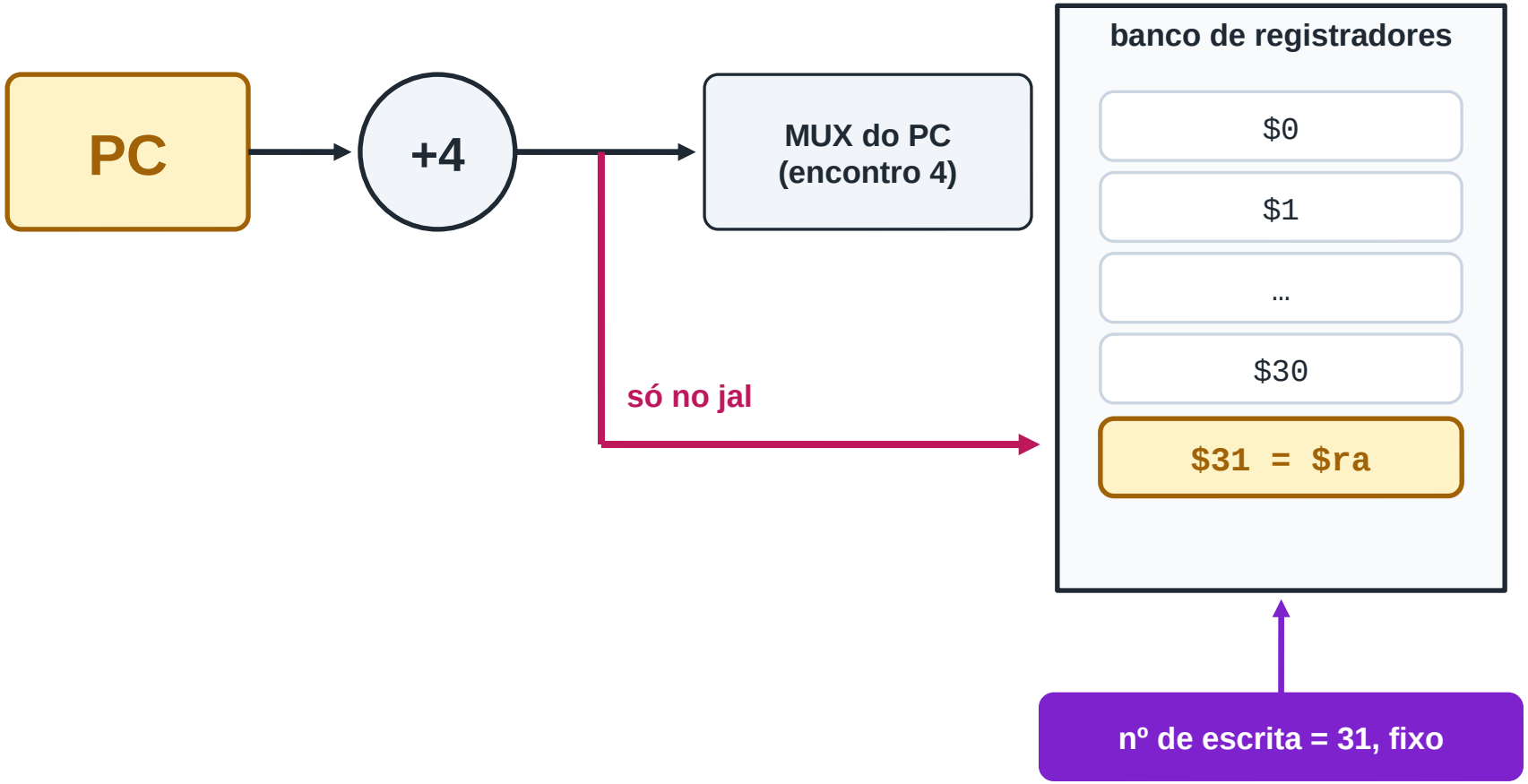
```
li   $v0, 4
la   $a0, msg1
syscall
li   $v0, 1
move $a0, $t1
syscall
…
li   $v0, 4
la   $a0, msg2
syscall
li   $v0, 1
move $a0, $t2
syscall
```

Para escrever o trecho uma vez só e usá-lo de vários lugares, falta resolver quatro coisas:

1. **Ir** até o trecho — **j já faz**
2. **Voltar** para o lugar certo — que muda a cada uso — **j não sabe**
3. **Levar** os dados que o trecho vai usar
4. **Trazer** o resultado de volta

Em C, tudo isso tem um nome só: função. Hoje vemos o que o compilador faz por trás de cada chamada.

jal é um fio a mais: de PC + 4 até o registrador 31



Na disciplina: um registrador guarda um valor quando o sinal de escrita manda. **Aqui:** no jal, o valor de PC + 4 — que o circuito já calculou — é escrito no registrador 31. Não há conta nova: só um caminho novo.

Uma função em C, peça por peça

```c
int soma_vetor(int v[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s = s + v[i];
    return s;
}

soma = soma_vetor(vetor, 8);
```

nome	vira um rótulo
parâmetros	chegam em $a0, $a1, $a2, $a3
return	o resultado sai em $v0
chamar	`jal soma_vetor`
voltar	`jr $ra`

Nenhuma dessas peças existe no processador. Todas são combinadas entre quem chama e quem é chamado — é uma convenção, como os nomes dos registradores no encontro 2.

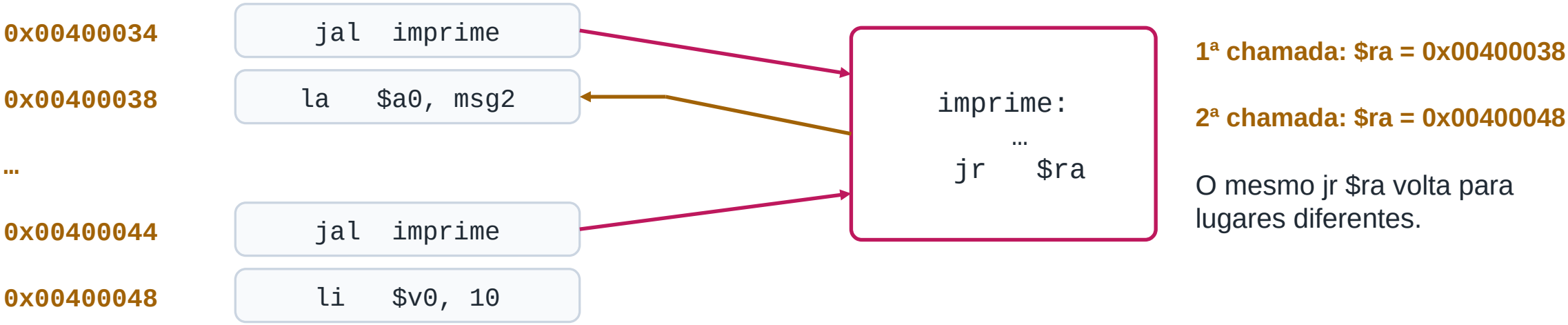
Um salto que lembra de onde saiu

`jal imprime` — **`$ra ← PC + 4 · PC ← imprime`**

jump and link: salta e anota a volta

`jr $ra` — **`PC ← $ra`**

jump register: salta para o endereço que está no registrador



1ª chamada: $ra = 0x00400038

2ª chamada: $ra = 0x00400048

O mesmo jr $ra volta para lugares diferentes.

Por que PC + 4? A volta é para a instrução seguinte ao jal — senão a função seria chamada de novo, para sempre.

jal é um j com uma anotação

`jal imprime → 0x0C10002A`



000011	00 0001 0000 0000 0000 0010 1010
3 · jal	0x010002A × 4 = 0x004000A8 = imprime

Formato J, igual ao j do encontro 4 — só o op muda: 2 é j, 3 é jal.

`jr $ra → 0x03E00008`



000000	11111	00000	00000	00000	001000
R	31 · $ra				8 · jr

Todo jr $ra do programa tem o mesmo código, 0x03E00008. O destino não está na instrução: está no registrador. É isso que permite voltar para lugares diferentes.

A convenção: quem leva, quem traz

Registradores	Uso combinado	No programa da aula
`$a0 – $a3`	até 4 argumentos, na ordem	endereço do vetor, n
`$v0 – $v1`	resultado da função	a soma, o maior
`$ra`	endereço de volta (escrito pelo jal)	onde main continua

```
la   $a0, vetor      # 1º argumento
li   $a1, 8          # 2º argumento
jal  soma_vetor
move $s0, $v0        # resultado
```

Antes do jal, quem chama põe os argumentos em $a.
Antes do jr, a função põe o resultado em $v0.

Mais de 4 argumentos vão para a pilha — não vamos precisar.

Promessa do encontro 2 cumprida: as regras de $a, $v e $ra "só fazem sentido com funções".

A mesma função, nas duas línguas

```c
int soma_vetor(int v[], int n)
{
    int s = 0;
    int i = 0;
    while (i < n) {
        s = s + v[i];
        i++;
    }
    return s;
}
```

```mips
soma_vetor:          # a0 = v, a1 = n
      li   $v0, 0           # s = 0
      li   $t0, 0            # i = 0
sv_laco: slt $t1, $t0, $a1
      beq  $t1, $zero, sv_fim
      lw   $t2, 0($a0)       # v[i]
      add  $v0, $v0, $t2
      addi $t0, $t0, 1       # i++
      addi $a0, $a0, 4       # próxima
      j    sv_laco
sv_fim:  jr   $ra
```

O que o C esconde: variáveis locais viram registradores $t; v[i] vira lw com o endereço andando de 4 em 4; return s vira “deixar s em $v0” + jr $ra. Os nomes s, i e v não existem mais.

O main ficou curto: só chama

```
main:
      la   $a0, vetor
      li   $a1, 8
      jal  soma_vetor
      move $s0, $v0      # soma

      la   $a0, vetor
      li   $a1, 8
      jal  maior_vetor
      move $s1, $v0      # maior

      la   $a0, msg1
      move $a1, $s0
      jal  imprime
      …   o mesmo para msg2 e $s1
```

soma_vetor	(v, n) → soma
maior_vetor	(v, n) → maior
imprime	(texto, valor) → nada

Run I/O

```
Soma = 41
Maior = 9
```

Mesmo resultado do encontro 4. A diferença está na organização: cada função faz uma coisa, e imprime é escrita uma vez e usada duas.

Depois de um jal, o que sobrevive?

`$t0 – $t9` temporários

A função chamada pode usar à vontade. Se quem chama precisa do valor depois, o problema é dele.

`$s0 – $s7` salvos

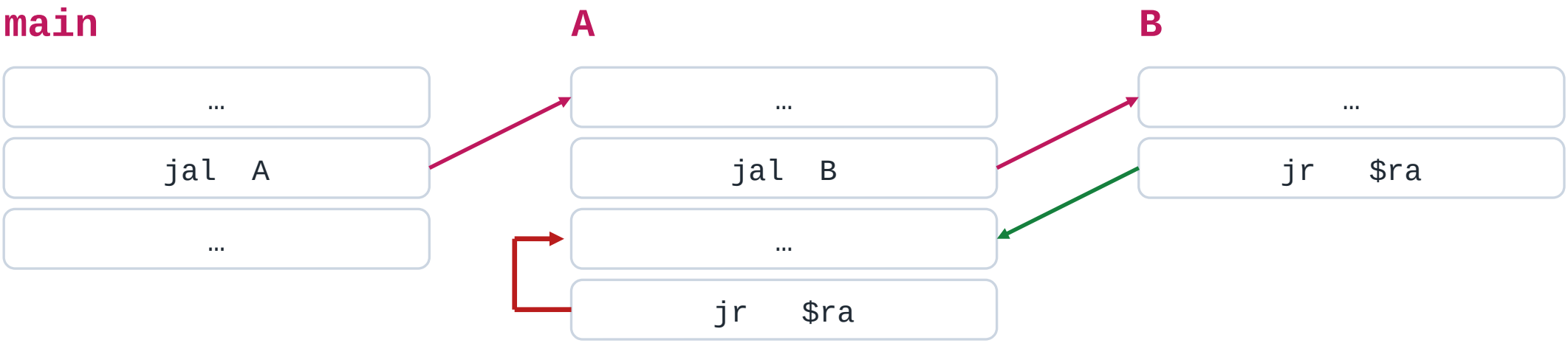
A função chamada precisa devolvê-los intactos. Se quiser usar um, guarda antes e restaura no fim.

```
jal  soma_vetor
move $s0, $v0      # a soma precisa sobreviver
jal  maior_vetor      # usa $t0–$t2
```

Por isso o main guarda os resultados em $s0 e $s1: as funções usam $t0, $t1 e $t2 como bem entendem.

Convenção, de novo: para o circuito, $t0 e $s0 continuam idênticos (encontro 2). A regra é um acordo — e a atividade mostra o que acontece quando alguém o quebra.

E quando uma função chama outra?



`jal A`	**$ra = volta para main**
`jal B`	**$ra = volta para A — o endereço de main foi perdido**
`jr $ra em B`	**volta para A: certo**
`jr $ra em A`	**volta para dentro de A, de novo e de novo**

Só existe um $ra. Quem chama outra função precisa guardar o seu antes.

Uma região da memória que cresce para baixo



empilhar

```
addi $sp, $sp, -8
sw   $ra, 4($sp)
sw   $a0, 0($sp)
```

desempilhar

```
lw   $a0, 0($sp)
lw   $ra, 4($sp)
addi $sp, $sp, 8
```

$sp aponta para o topo da pilha. Ela começa no alto da memória (encontro 3) e cresce para endereços menores. O último a entrar é o primeiro a sair — como uma pilha de pratos.

Deslocamento negativo, prometido no encontro 3: o addi $sp, $sp, −8 é o imediato em complemento de dois.

Prólogo e epílogo: abrir e fechar o quadro

```
funcao:
      addi $sp, $sp, -8      # abre espaço
      sw   $ra, 4($sp)       # guarda a volta
      sw   $a0, 0($sp)       # guarda o n

      …   corpo: pode fazer jal à vontade   …

      lw   $ra, 4($sp)       # recupera a volta
      addi $sp, $sp, 8       # fecha o espaço
      jr   $ra
```

prólogo

no começo: baixa o $sp e guarda $ra e o que mais a função precisar reencontrar depois de um jal (argumentos, $s que vá usar).

epílogo

no fim: recupera o que guardou e sobe o $sp exatamente o mesmo tanto.

folha

quem não chama ninguém não precisa de quadro.

O $sp precisa terminar a função exatamente onde começou. Se sobrar ou faltar 4 bytes, quem chamou encontra a pilha bagunçada.

A função que chama a si mesma

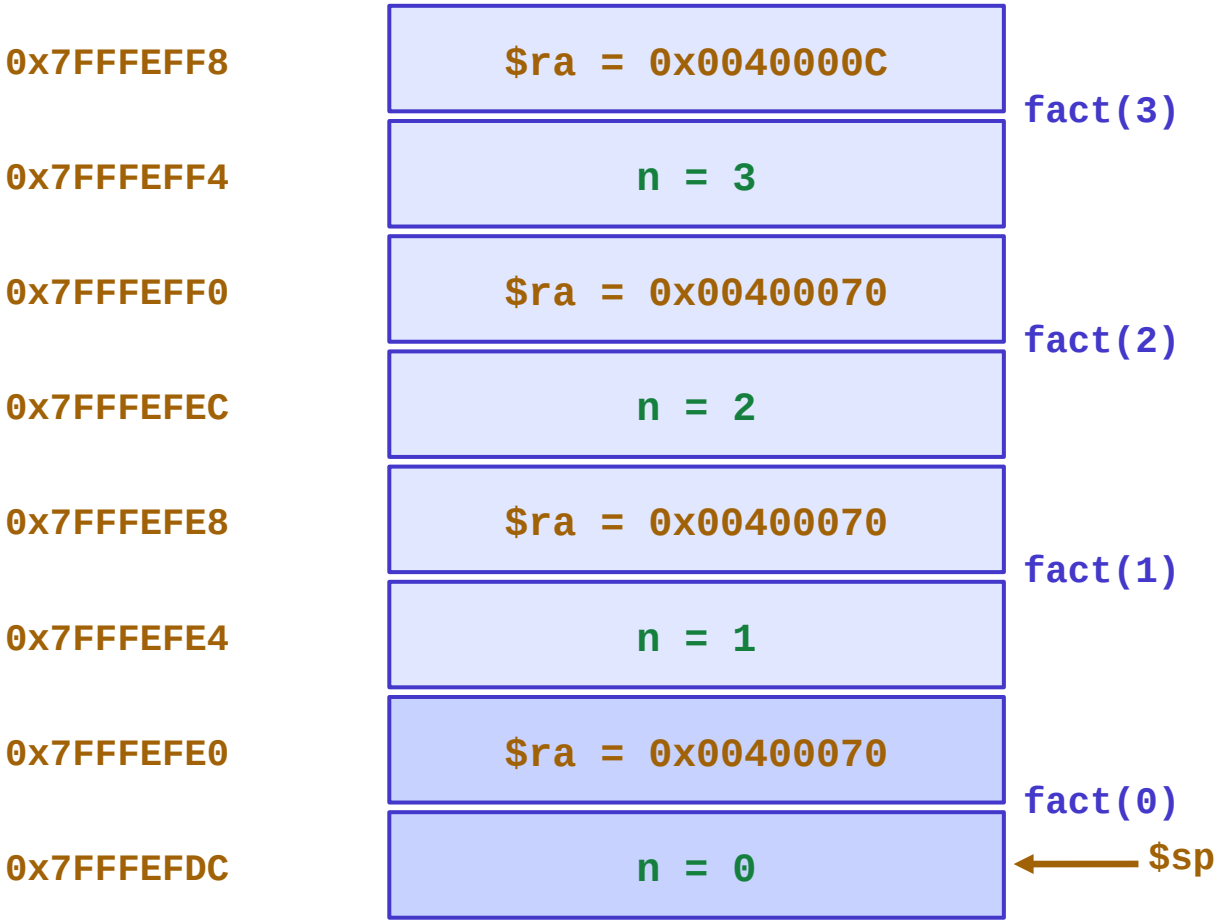
```
int fact(int n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
```

`fact(3) = 3 · fact(2) = 3 · 2 · fact(1) = 3 · 2 · 1 · fact(0) = 6`

Por que guardar o n? A chamada de dentro usa $a0 para o seu próprio n − 1. Quando ela volta, o n desta chamada precisa ser lido de onde ficou guardado.

```
fact:
      addi $sp, $sp, -8
      sw   $ra, 4($sp)
      sw   $a0, 0($sp)
      bne  $a0, $zero, geral
      li   $v0, 1            # fact(0) = 1
      j    sai
geral: addi $a0, $a0, -1     # n - 1
      jal  fact           # $v0 = fact(n-1)
      lw   $a0, 0($sp)       # o n desta chamada
      mul  $v0, $a0, $v0     # n · fact(n-1)
sai:   lw   $ra, 4($sp)
      addi $sp, $sp, 8
      jr   $ra
```

No fundo da recursão, quatro quadros empilhados



Na volta, cada quadro desfaz o seu:

`fact(0)`	devolve 1
`fact(1)`	lê n = 1: 1 · 1 = 1
`fact(2)`	lê n = 2: 2 · 1 = 2
`fact(3)`	lê n = 3: 3 · 2 = 6 → main

Três quadros guardam o mesmo $ra, 0x00400070: a instrução depois do jal fact de dentro. Só o primeiro volta para o main.

Cada linha de C, o que ela custa em MIPS

Em C	Em MIPS	Quem cuida
`f(x, y)`	argumentos em $a0, $a1; jal f	quem chama
`return r;`	r em $v0; jr $ra	a função
`variável local`	um registrador $t — ou espaço no quadro	a função
`variável que atravessa uma chamada`	um $s — ou espaço no quadro	quem chama
`f chama g`	guardar $ra no quadro (prólogo/epílogo)	f
`recursão`	nada de especial: cada chamada tem o seu quadro	—

O compilador segue exatamente estas regras. É por isso que uma função em C, compilada por um, pode chamar outra compilada por outro — e até uma escrita à mão em assembly.

Exercício invertido: qual foi o C?

```
f:    slt  $t0, $a0, $a1
      beq  $t0, $zero, f_a
      move $v0, $a1
      jr   $ra
f_a:  move $v0, $a0
      jr   $ra
```

```
int ______(int a, int b) {
    if (________)
        return ___;
    return ___;
}
```

1. Quantos parâmetros f recebe? Como você sabe?
2. f devolve alguma coisa? O quê?
3. Escreva f em C — e dê a ela um nome melhor.

Pistas: quais $a são lidos sem serem escritos antes? O que fica em $v0 antes de cada jr?

Atividade

1	**Observar** — acompanhar o $ra e o PC durante os jal e jr do aula5_funcoes.asm	7 min
2	**Reconstruir** — exercício invertido: escrever em C a função que gerou um assembly	10 min
3	**Empilhar** — ver os quadros de fact(3) na janela Data Segment	7 min
4	**Provocar** — quebrar a convenção duas vezes — e explicar o que aparecer	6 min

O enunciado está na folha de atividade. O molde de prólogo e epílogo e a tabela de convenções estão na folha de consulta.

Duas convenções quebradas

`move $t0, $v0   (em vez de $s0)`

```
Run I/O:

Soma = 8
Maior = 9
```

maior_vetor usou $t0 como i. Quando voltou, $t0 valia 8 — o n. A soma tinha sido apagada sem erro nenhum.

`sem sw $ra e lw $ra no fact`

```
Runtime exception at 0x00400074:
arithmetic overflow
```

fact(5) nunca aparece. Cada jr $ra volta para dentro de fact, que sobe o $sp mais 8 — até o $sp passar de 0x7FFFFFFF e a addi transbordar.

Nos dois casos, nenhuma instrução estava errada. O que falhou foi o acordo entre quem chama e quem é chamado.

O que fica de hoje

1. jal salta e guarda a volta em $ra. jr $ra volta.
2. Argumentos vão em $a0–$a3; o resultado volta em $v0.
3. Depois de um jal, $t pode ter mudado; $s não.
4. Quem chama outra função guarda $ra na pilha: prólogo e epílogo.
5. A pilha cresce para baixo; o $sp termina onde começou. Recursão é só isso.

Função não é um recurso do processador. É um acordo.

Agora todos os enunciados estão liberados

Exercício	Enunciado	Pede
`A.6`	Calculadora que acumula inteiros até receber zero	laço
`A.7`	Soma dos dois maiores entre três valores lidos	if
`A.9`	Os cem primeiros números primos, com sub-rotina de primalidade	função
`A.10`	Torres de Hanói, recursivo	pilha
`livre`	Conversor entre bases de numeração	laço

README.md
o que faz e como executar

Repositório da liga
código publicado

Encontro 6
cada dupla apresenta

Dica para o A.9: escrevam primeiro a função eh_primo(n) e testem sozinha. Só depois o laço que conta até cem primos.

Enunciados: Patterson & Hennessy, Apêndice A, Seção A.12.

21 · SEMANA QUE VEM

Do simulador ao silício

Desde o encontro 1, todos os programas rodaram num simulador. Na semana que vem, um programa escrito por vocês roda num processador de verdade, implementado numa placa FPGA, aqui no laboratório.

- Lá não há sistema operacional. Quem atende o syscall?
- Sem Run I/O, como saber se o programa funcionou?
- Todas as instruções que vocês usaram existem lá?

Na aula 6: tragam o projeto — e as respostas.